

SD	Sistemas Distribuidos
26/27	Práctica no guiada: Sockets, Streaming de Eventos, Colas y modularidad.
	Water Management Network

Preámbulo

El objetivo de esta práctica es que los estudiantes extiendan y afiancen sus conocimientos sobre el uso de la tecnología de comunicación básica sockets, estudiada durante las sesiones de teoría.

Los sockets son la base de las comunicaciones entre computadores y suponen el punto de enlace básico entre nuestra aplicación e Internet, utilizando muy pocos recursos de red. Como contrapartida, las aplicaciones que utilizan sockets como mecanismo de comunicación deben interpretar los mensajes que intercambian entre sí, es decir, deben establecer un protocolo o acuerdo de comunicación entre ellos. Esta necesidad implica un fuerte acoplamiento entre las aplicaciones distribuidas que utilizan sockets como mecanismo de comunicación, ya que todas las partes deben conocer de antemano la estructura de los mensajes que serán intercambiados y codificarlo en su programación.

Por otro lado, otras tecnologías y arquitecturas actuales orientadas al streaming de eventos en tiempo real, basadas en sockets pero de más alto nivel en términos de construcción y desarrollo, permiten la implementación de soluciones menos acopladas y más resilientes que estos.

Esta práctica establece el marco inicial de un desarrollo que se ampliará durante el cuatrimestre y que permitirá al estudiante crear una aplicación similar a las que se desarrollan hoy en día en los entornos empresariales, poniendo el foco en el uso e integración de distintos paradigmas de comunicación susceptibles de ser utilizados.

El riego inteligente en las ciudades

La gestión eficiente del agua se ha convertido en uno de los grandes retos de las ciudades actuales. El mantenimiento de parques, jardines y zonas verdes urbanas requiere de un riego constante y bien planificado, pero este consumo debe conciliarse con la necesidad, cada vez más acuciante, de un uso racional y sostenible del agua.

El riego manual o no supervisado presenta importantes ineficiencias: exceso de riego en zonas con humedad suficiente, fugas no detectadas que se prolongan durante horas o días sin que nadie lo advierta, o estaciones fuera de servicio de las que no se tiene constancia hasta que el personal de mantenimiento las visita físicamente. Frente a esto, una red de estaciones de riego conectadas y supervisadas de forma centralizada permite conocer en todo momento el estado de cada punto de la ciudad, detectar incidencias de forma temprana, evitar el derroche de un recurso escaso y facilitar la planificación del mantenimiento de las zonas verdes.

Además, la monitorización continua de humedad y consumo aporta información valiosa para la toma de decisiones tanto técnicas como económicas y medioambientales, y sienta las bases para una futura optimización del riego basada en datos reales de cada zona.

Este no es un problema teórico: en España, las pérdidas reales de agua en las redes de distribución (fugas, roturas y averías) se estiman en un 14,6% del volumen total suministrado (INE, 2024). Alicante, en cambio, se sitúa muy por encima de esa media, con un rendimiento técnico hidráulico del 88,55%, apoyado en la digitalización de su red de sensores y contadores. La propia ciudad cuenta además con una red de agua regenerada que da acceso a riego a cerca del 87% de sus zonas verdes, reduciendo así el consumo de agua potable destinado a esta finalidad.

En este contexto, la práctica que se propone busca acercar al estudiante al reto de implementar los conceptos teóricos de la asignatura de sistemas distribuidos en una problemática real: gestionar y supervisar de manera integral, resiliente, escalable y segura una red de riego que cubre múltiples parques y jardines de una ciudad.

Especificación

El objetivo de la práctica es desarrollar un sistema distribuido que implemente una simulación de una solución para la gestión centralizada del riego de los parques y jardines de una ciudad.

Descripción funcional

Los estudiantes deberán implementar una solución de control online en tiempo real denominada **"WaterManagement"**.

Alcance

La solución tendrá como objetivo realizar un sistema que simula la gestión de una red de estaciones de riego (en adelante también **"Watering Station"** o **"WS"**) distribuidas por los parques y jardines de una ciudad, y su uso tanto por el ayuntamiento/gestor de la red como por los operarios de mantenimiento que activan y supervisan el riego sobre el terreno.

2 componentes

Nuestra simulación consistirá en, partiendo de un mapa de una ciudad ficticia, disponer de una serie de estaciones de riego repartidas entre sus distintos parques y jardines, en las que los operarios, provistos de una aplicación, podrán activar y supervisar el riego de cada zona.

Todas las estaciones de riego son gestionadas desde una central que las dará de alta, las operará y las monitorizará en caso de que se produzcan incidencias.

Para la realización de todo el sistema, nos basaremos en una serie de componentes distribuidos que estarán interconectados y que conformarán la solución de manera resiliente, escalable y segura.

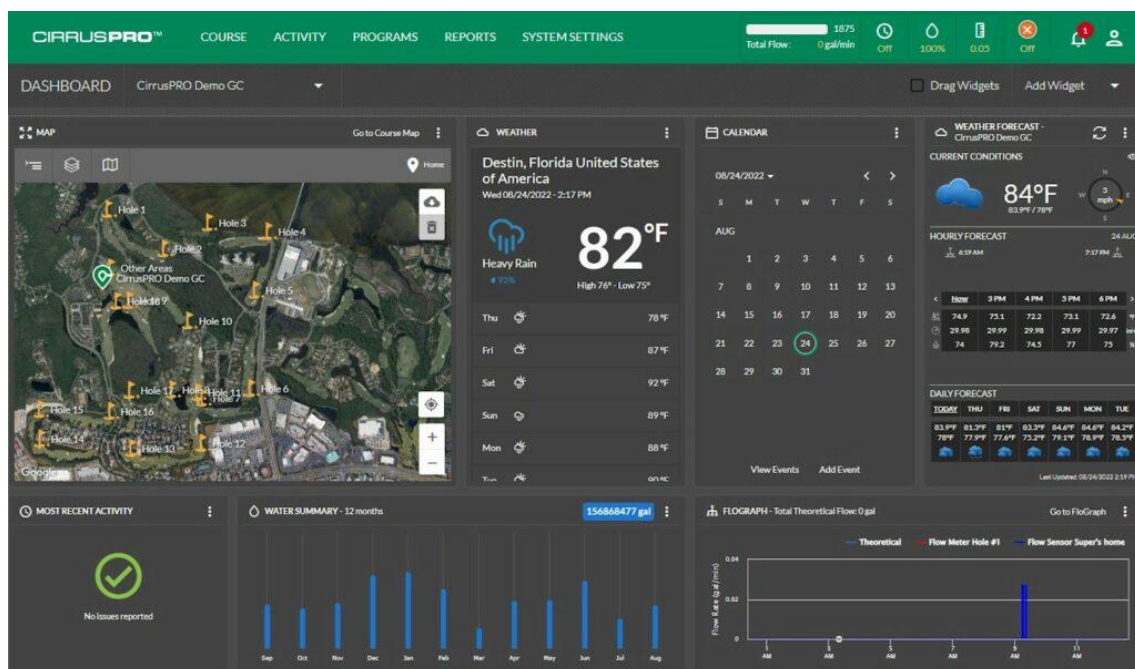


Imagen 1: Sistemas de riego de la empresa Rain Bird

CENTRAL. Sistema de monitorización de la red

Dispondremos de un cuadro de mandos o **panel de monitorización** donde se podrá observar el detalle de funcionamiento en **tiempo real** de todo el sistema incluyendo los estaciones de riego, **las peticiones de activación y el estado del sistema central**.

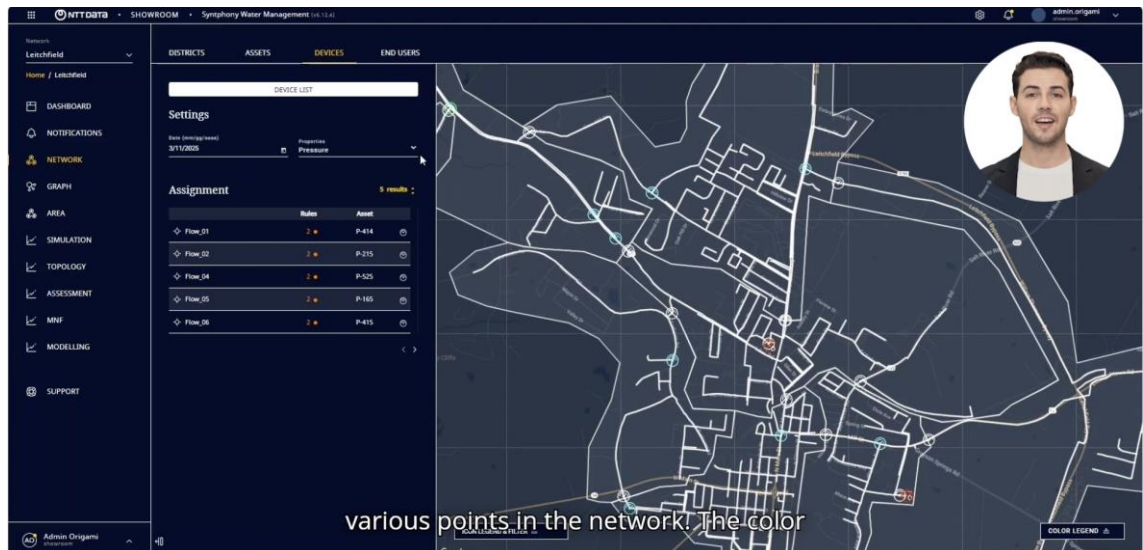


Imagen 2: Ejemplo de sistema de monitorización de una red de agua

En el panel que muestre la información de detalle la **información a visualizar** será la siguiente:

- ID de la estación de riego.
- Ubicación de la estación (parque/jardín/zona de la ciudad).
- Cantidad de agua que se está suministrando. ?
- Estado:
 - **Disponible (activada)**: La WS está funcionando correctamente y a la espera de una solicitud de riego. Se mostrará mediante un fondo de color **VERDE**.
 - **Regando**: La WS está funcionando correctamente y está, en ese momento, suministrando riego a la zona que cubre. Se mostrará mediante un fondo de color **VERDE PARPADEANDO** y con los siguientes datos impresos:
 - Caudal en litros/minuto en tiempo real.
 - Volumen acumulado en litros en tiempo real.
 - Id del operario que ha activado el riego.
 - **Fuga**: La WS está conectada al sistema central, pero presenta una fuga detectada por su sensor y no debe producirse riego hasta resolver la avería. Se mostrará mediante un color **ROJO** de fondo.
 - **Fuera de servicio (parada)**: La WS está en correcto funcionamiento, pero no puede usarse deliberadamente por orden de CENTRAL. Se mostrará mediante un fondo de color **NARANJA** con la leyenda "Fuera de Servicio".
 - **Desconectada**: La WS no está conectada al sistema central (por avería, fallo de red o cualquier otro motivo) y no es posible monitorizar su uso. Se mostrará mediante un color **GRIS** de fondo.

OPERARIOS DE CAMPO (FIELD OPERATORS – FO)

Los operarios (personal de mantenimiento de parques y jardines) dispondrán de una aplicación que les permitirá activar el riego de una estación determinada. Dicha aplicación enviará a CENTRAL la petición de activación y ésta le responderá autorizándola o denegándola.

El sistema central, como se verá en el apartado de mecánica de la solución, deberá velar porque la estación de riego esté disponible antes de autorizar la activación.

ESTACIONES DE RIEGO (WATERING STATION - WS)

Estaciones ubicadas en los distintos parques y jardines de la ciudad en las que se realiza el riego de la zona verde que cubren.

Cada estación de riego integra:

- **Caudalímetro:** mide la cantidad de agua que está suministrando.
- **Detector de fuga:** detecta pérdidas anómalas de agua en la instalación de la estación.
- **Actuador de electroválvula:** permite abrir o cerrar el paso de agua para iniciar o detener el riego.

como se
detecta una
fuga?

Los estados en los que puede entrar una WS son los expresados anteriormente en el panel de control de CENTRAL.

Las estaciones de riego permiten las siguientes funcionalidades:

- Registrarse en CENTRAL: enviarán para ello su Id y ubicación a CENTRAL.
- Activarse o pararse.
- Regar: proporcionar el servicio de riego de **dos formas:**
 - Manualmente, mediante una opción en la propia estación.
 - A través de una petición proveniente de la aplicación del operario (vía CENTRAL, como se verá más adelante) o incluso desde la propia CENTRAL.
- Medir el volumen de agua que están suministrando.

mediante comunicación socket

Estas estaciones disponen a su vez de una capacidad de supervisión continua de su propio estado (sensores y actuador) que permite controlar su buen funcionamiento o detectar una posible avería.

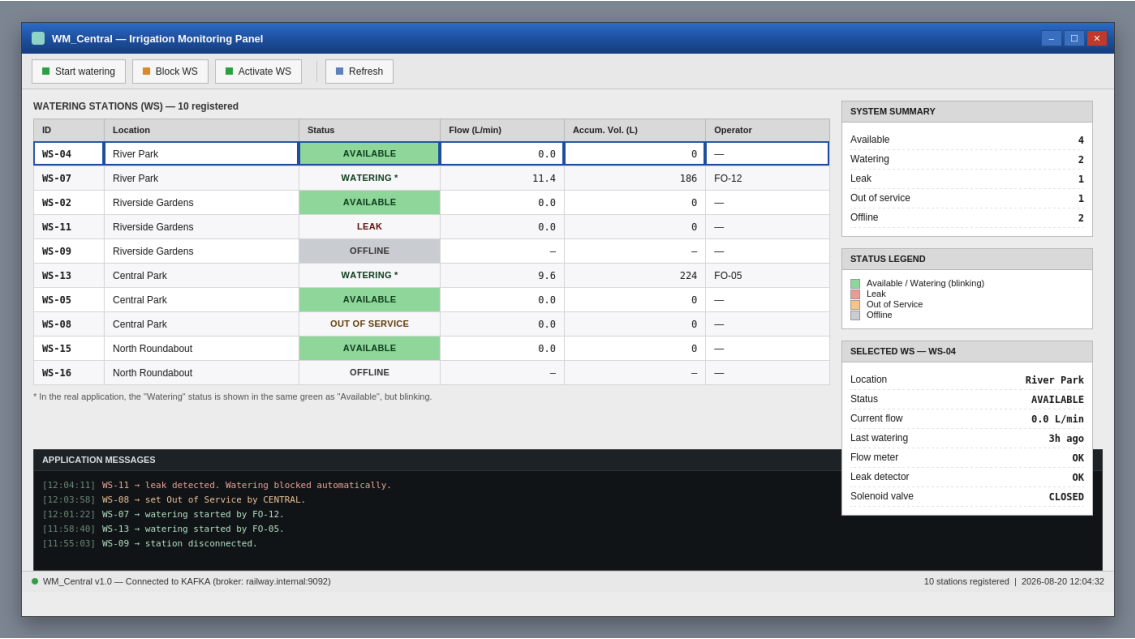


Figura 1: Ejemplo de panel de control del sistema central en versión de aplicación escritorio.

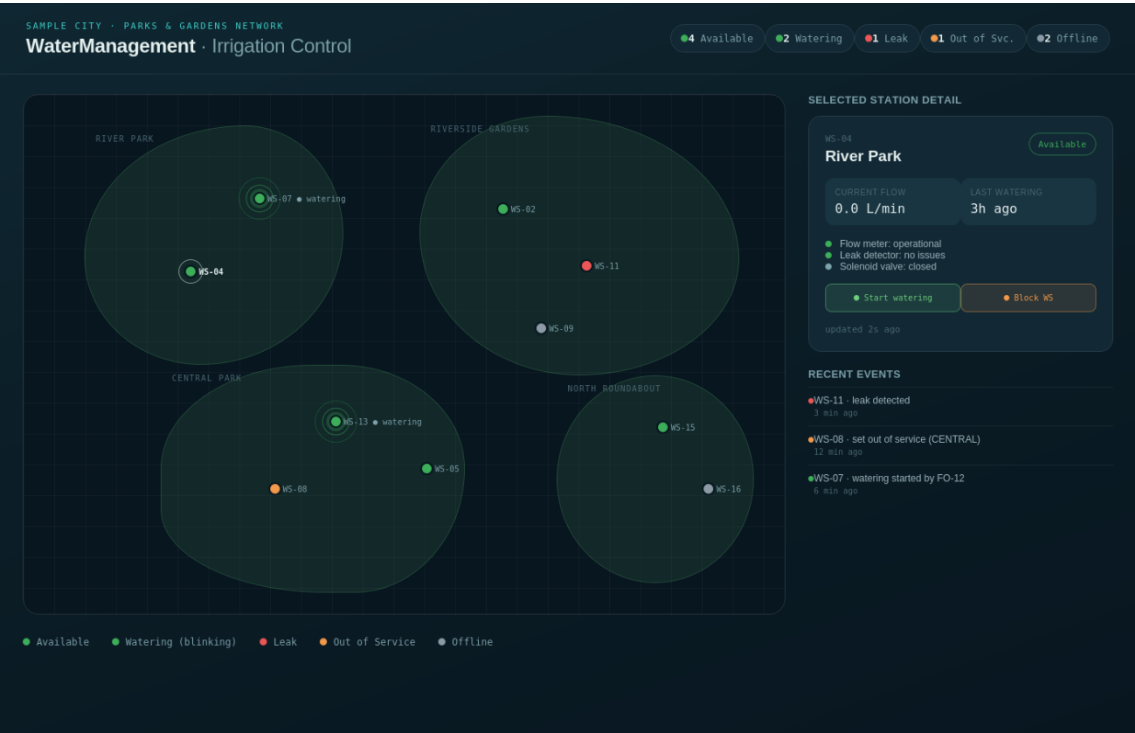


Figura 2: Ejemplo de panel de control del sistema central en versión de front web.

Mecánica de la solución

El sistema comienza mediante la **ejecución del módulo central** (en adelante "CENTRAL") que **permanecerá en ejecución sin límite de tiempo sin apagarse en ningún momento** (salvo que una excepción o bloqueo súbito provoque su apagado no controlado).

Tendrá lugar entonces la siguiente secuencia:

1. Ante cualquier ejecución o reinicio, CENTRAL **comprobará** (en su BD) si ya tiene estaciones de riego disponibles registradas (con su ubicación) y las mostrará en su panel de monitorización y control en su estado correspondiente. **IMPORTANTE:** **hasta que una estación de riego no conecte con CENTRAL, ésta no podrá conocer el estado real de la estación. En ese caso, la mostrará con el estado DESCONECTADA.**

2. CENTRAL siempre estará a la espera de:

- Recibir peticiones de registro y alta de una nueva estación de riego.
- Recibir peticiones de activación de un riego.

3. Los operarios, desde su aplicación o desde un menú en la propia estación, solicitarán la activación del riego en cualquier estación. Con el objetivo académico de automatizar las pruebas del sistema, además de poder solicitar la activación manualmente, la aplicación del operario también podrá leer las activaciones a solicitar desde un archivo con el siguiente formato

4. CENTRAL procederá a realizar las comprobaciones oportunas para validar que la estación de riego esté disponible (activada, sin avería ni fuga detectada) y, en su caso, solicitará autorización a la estación para que proceda al riego. Todo el proceso requerirá de la notificación al operario de los pasos que van sucediendo hasta autorizar o denegar el riego. Dichos mensajes se deben mostrar claramente en pantalla, tanto en la aplicación del operario como en CENTRAL.

5. Las estaciones de riego, una vez se han registrado y conectado a CENTRAL, estarán en estado de reposo a la espera de que un operario solicite, bien en el propio interfaz de la estación o a través de su aplicación, una activación de riego.

6. Realizadas todas las comprobaciones por CENTRAL y enviada la notificación tanto a la WS como a la aplicación del operario para que se proceda a la activación, el sistema procederá a la apertura de la electroválvula.

7. Durante el riego, la estación de riego estará enviando información constante (cada segundo) a CENTRAL, la cual mostrará en su panel de monitorización el caudal y el volumen acumulado que va teniendo lugar en cada WS, tal y como se indicó en apartados anteriores. La aplicación del operario igualmente mostrará esos estados de la WS que está regando.

8. **El riego continuará indefinidamente** hasta que se produzca una de estas dos condiciones:

- Se alcance un límite de tiempo preestablecido en el momento de iniciar el riego.
- Se ejecute una opción de menú en la WS para finalizar el suministro.

En ambos casos las WS lo notificará a CENTRAL, quien a su vez enviará un resumen final del riego (volumen total consumido, duración, etc.) al operario. La WS volverá al estado de reposo a la espera de una nueva petición de activación.

9. Durante todo el funcionamiento del sistema, la estación de riego estará comprobando el estado de salud de sus propios elementos. En caso de avería (detección de una fuga), notificará

a CENTRAL dicha situación. Una vez resuelta la contingencia, se enviará un nuevo mensaje a CENTRAL y la WS volverá a estar disponible. Si la avería se produjera durante un riego en curso, éste debe finalizar inmediatamente informando a todos los módulos y aplicaciones de dicha situación. Toda la información tiene que ser claramente visible en pantalla tanto en CENTRAL como en el monitor de la estación.

10. CENTRAL recibirá todos los estados y riegos de todas las WS. Los operarios sólo recibirán los mensajes de la WS específica que les esté prestando servicio, pero también podrán ver todas las WS que estén disponibles.

11. CENTRAL permanecerá siempre en funcionamiento a la espera de nuevas peticiones de activación. Adicionalmente dispondrá de opciones para, de forma arbitraria, poder enviar a una o todas las WS cualquiera de estas órdenes:

- **Iniciar un riego:** Se comportará de la misma forma que si lo hiciera un operario.
- **Bloquear la WS:** La WS finalizará cualquier riego si estuviera en ejecución y pondrá su estado en NARANJA mostrando el mensaje de "Fuera de Servicio".
- **Activar la WS:** La WS volverá a estado ACTIVADA y pondrá su color en VERDE.

La acción se va desarrollando de forma asíncrona, es decir, sin "turnos", de manera que cada operario, CENTRAL y cada WS envían mensajes o peticiones de riego en cualquier momento.

Diseño técnico

Se propone al estudiante implementar un sistema distribuido compuesto, al menos, de los siguientes componentes software:

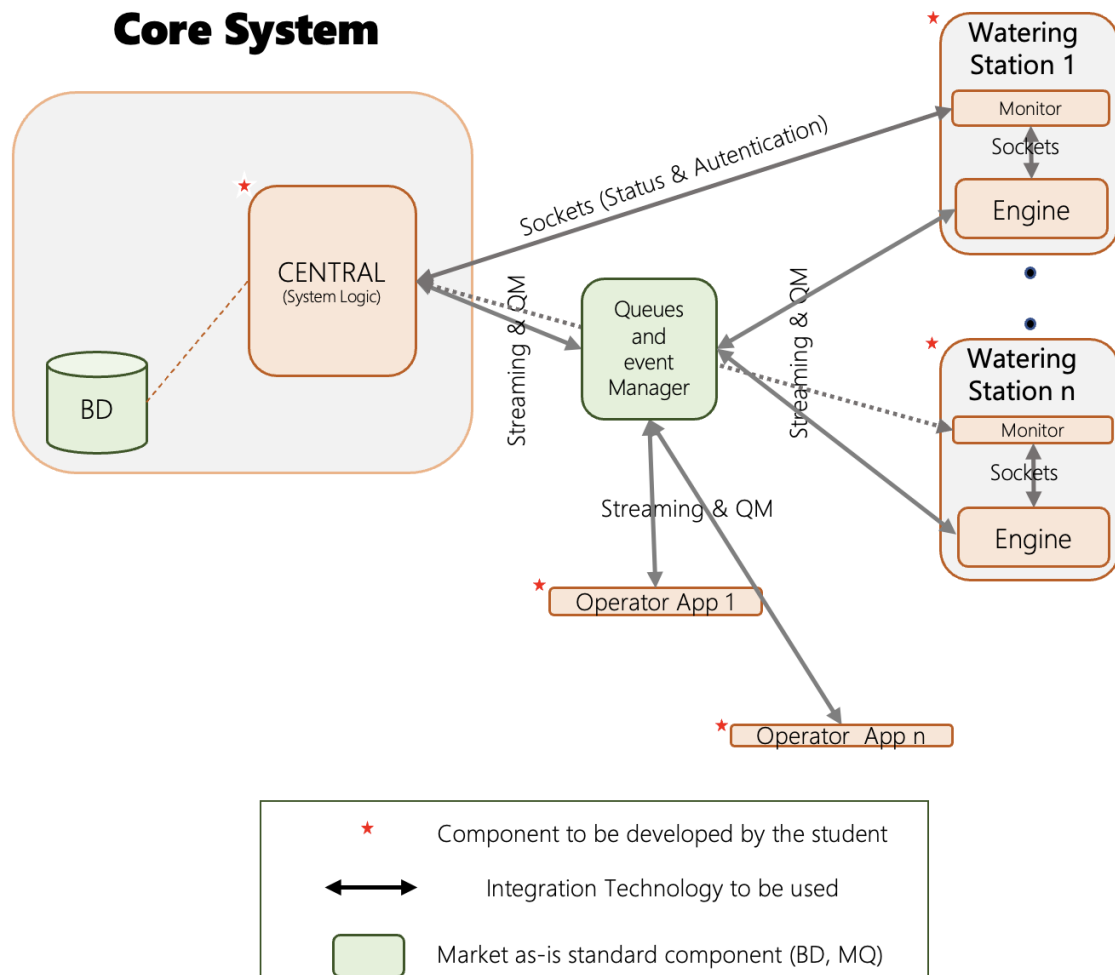


Figura 3. Arquitectura conceptual del Sistema software, interconexiones entre los componentes y tipos de interfaces. Release 1

Los componentes software que el estudiante ha de desarrollar son los siguientes:

- **Core System:** Módulos centrales de la solución.
 - **CENTRAL:** Módulo que representa la central de control de toda la solución. Implementa la lógica y gobierno de todo el sistema.
- **Watering Station (WS):** Módulos que simulan las estaciones de riego.
 - **Engine:** Módulo que recibe la información del caudalímetro y activa la electroválvula.
 - **Monitor:** Módulo que monitoriza la salud de todo el sistema y que reporta a la CENTRAL cualquier avería de este (fuga). Sirve igualmente para autenticar y registrar a los WS en la central cuando sea oportuno.
- **Operator App:** Aplicación que usan los operarios (jardineros) para controlar las estaciones de riego.

Los componentes pueden ser desarrollados en el lenguaje de preferencia del estudiante: Java, C/C++, . NET, Python, etc. asumiendo el estudiante la responsabilidad de su conocimiento y forma de desplegarlo.

El subsistema de gestión de streaming de eventos será implementado mediante la tecnología KAFKA cuyo funcionamiento se explicará en las sesiones de prácticas.

A continuación, se especifica más detalladamente cada componente.

Core System

Contiene todos los módulos que implementan la estructura principal del sistema.

CENTRAL:

Se trata de la aplicación que implementará la lógica fundamental del sistema. El nombre de la aplicación será **obligatoriamente “WM_Central”**. Para su ejecución recibirá por la línea de parámetros al menos los siguientes argumentos:

- Puerto de escucha del servidor de sockets que ofrece al WS_WM_M.
- IP y puerto del Broker/Bootstrap-server del gestor de colas.

Al arrancar la aplicación quedará a la escucha en el puerto establecido e implementará todas las funcionalidades necesarias para ejecutar la mecánica de la solución expresada en el apartado anterior **incluidas las indicadas en el punto 11** del apartado “Mecánica de la solución”.

Base de Datos:

Contendrá, los datos de los operarios, WSs (Id, estado,...) así cualquier otra información que los estudiantes consideren necesaria para la implementación del sistema.

Los estudiantes usarán obligatoriamente SQLite para su implementación.

Watering Station (WS)

Engine:

Se trata de la aplicación que implementará la lógica principal de todo el sistema. El nombre de la aplicación será **obligatoriamente “WM_WS_E”**. Para su ejecución recibirá por la línea de parámetros al menos los siguientes argumentos:

- IP y puerto del Broker/Bootstrap-server del gestor de colas.
- IP y puerto del Monitor del WS

La aplicación permanecerá a la espera hasta recibir una solicitud de servicio procedente de la central. **La WS quedará asignada con el ID recibido en la línea de parámetros del WM_WS_M y cuya validez deberá ser confirmada por WM_Central en el proceso de autenticación.**

Monitor:

Aplicación que simula un módulo de gestión de observación de todo el WS (hardware y software) velando por la seguridad y correcto funcionamiento de este sin incidencias. El nombre de la aplicación será **obligatoriamente “WM_WS_M”**. Para su ejecución recibirá por la línea de parámetros al menos los siguientes argumentos:

- Puerto del servidor de sockets que ofrece a WM_WS_E
- IP y puerto del WM_Central
- ID de la WS: Identificador único con que esa WS está dada de alta en la red.

Al arrancar la aplicación, esta se conectará al WM_Central para autenticarse y validar que el WS está operativo y preparado para prestar servicios. De la misma forma, esperará la conexión del Engine del WS al que pertenece y empezará a **enviarle cada segundo un mensaje de comprobación de estado de salud**. Si no se recibe respuesta desde el WM_WS_E o se recibe una respuesta tipo KO, WM_WS_M enviará a WM_Central un mensaje de avería simulando una detección de fuga como se ha indicado en anteriores apartados. Para simular dichas incidencias, la aplicación WM_WS_E deberá permitir que, en tiempo de ejecución, se pulse una tecla para reportar un KO al monitor.

Field Operators (FO)

Aplicación que disponen los usuarios del sistema para solicitar un suministro. El nombre de la aplicación será **obligatoriamente “WM_FO”**. Para su ejecución recibirá por la línea de parámetros al menos los siguientes argumentos:

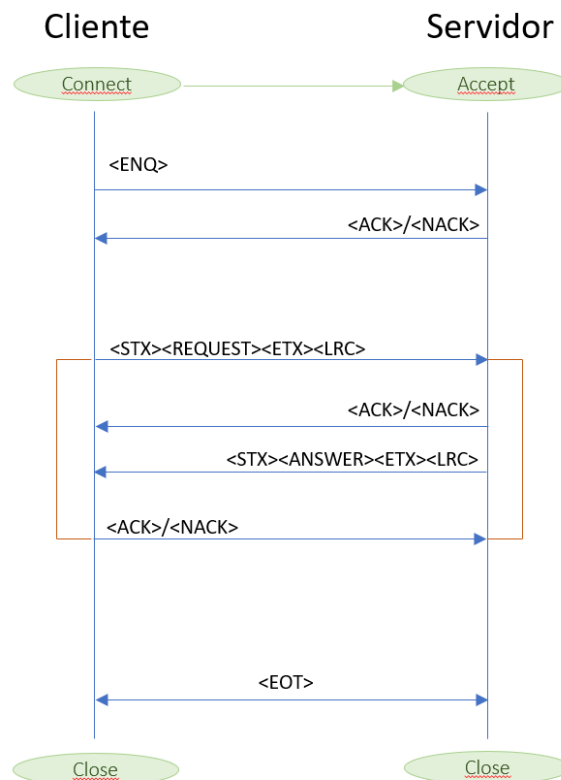
- IP y puerto del Broker/Bootstrap-server del gestor de colas.
- ID del operador: Identificador único con que ese operador está dado de alta en CENTRAL.

Esta aplicación, además de poder solicitar un suministro puntualmente en un WS, podrá leer un fichero todos los servicios que va a solicitar. Enviará el primero a WM_Central y, tras su conclusión (sea en éxito o fracaso), esperará 4 segundos y pasará a solicitar el siguiente servicio.

Protocolos de comunicación

Protocolo estándar sockets

Como se comenta al principio de este documento, la comunicación mediante sockets obliga a los actores involucrados a definir la mensajería y protocolo con precisión. Los estudiantes podrán definir los protocolos de comunicación entre los distintos módulos del sistema a su criterio. **Los profesores recomiendan, aunque no es obligatorio**, usar una mensajería basada en el estándar de empaquetado <STX><DATA><ETX><LRC> que corresponde al siguiente flujo:



Donde:

- <REQUEST> y <ANSWER>: Contienen el mensaje transmitido entre ambos puntos con los campos separados por un carácter determinado:
 - Ej.: <REQUEST>: Código Operación#campo1#...#campo n
- <LRC>: Se define como el XOR(MESSAGE) byte a byte y sirve para validar que la transmisión del mensaje se ha realizado satisfactoriamente y ha sido recibida de forma correcta por el destinatario el cual responderá al mismo con un <ACK> o <NACK>

Aclaraciones finales

Antes incongruencias funcionales o aspectos no considerados en los anteriores apartados que impidan la correcta implementación de la práctica, los estudiantes deberán

implementar la alternativa que entiendan más conveniente previa consulta con el profesor para consensuar dicha implementación evitando un excesivo nivel de complejidad.

Guía mínima de despliegue

Para la correcta evaluación de la práctica es necesario comprobar que la aplicación distribuida solicitada es desplegada en un entorno verdaderamente distribuido.

Se desplegará obligatoriamente de la siguiente manera:

- Sistemas en Cloud (alojados en RAILWAY www.railway.com) : WM_Central , BD y KAFKA
- Laboratorio: varios WM_FO en un PC y varios WM_WS en otro PC.

Al menos se ha de desplegar el siguiente escenario.

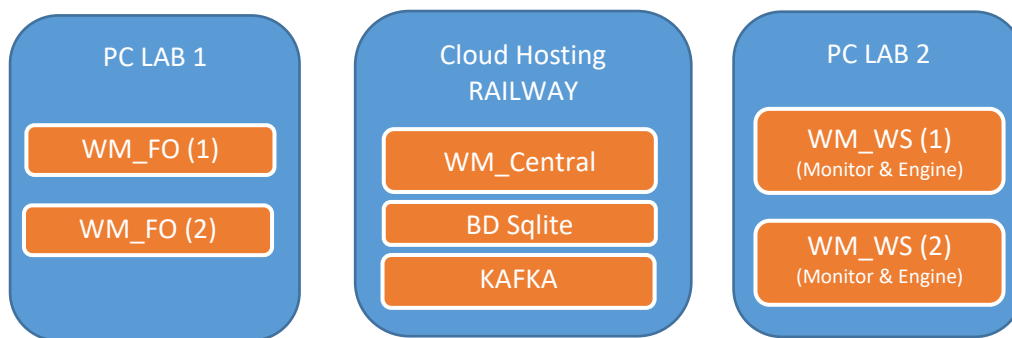


Figura 4. Escenario físico mínimo para el despliegue de la práctica.

- El alumno tendrá que asistir a la evaluación con **la parte Cloud totalmente desplegada y en funcionamiento. Solo desplegará en el laboratorio los módulos FO y WS.**
- **El uso de la tecnología Docker para el despliegue es obligatorio en todos los casos** para el despliegue de cualquier módulo.
- **El uso de Github es obligatorio para el manejo del repositorio de código.** Cada grupo deberá dar acceso a su profesor de prácticas a dicho repositorio.
- El ciclo de CI/CD deberá ser: VS Code -> Github -> Railway (Aconsejable por su simplicidad) o VS Code -> Github → AWS Learner Lab (Mayor configuración)

Entregables y evaluación

- La evaluación de la práctica se realizará en los laboratorios.
- **Se podrá realizar en grupos de hasta 2 personas sin perjuicio de que, durante el momento de la corrección, el profesor pueda preguntar a cualquier estudiante del grupo por cualquier aspecto de cualquiera de los módulos.**
- Los estudiantes deben desplegar por ellos mismos la práctica que resuelve el enunciado anterior. Deben desplegar un sistema completo, es decir, una central, los WSs (engine y monitor) y Field Operator todos ellos interconectados entre sí. **Este requisito es indispensable para poder realizar la corrección.**

- Deben poder evaluarse positiva o negativamente todos los apartados que aparecerán en la Guía de corrección que se entregará a tal propósito.

Sobre el uso de herramientas de Inteligencia Artificial (IA)

- El uso de asistentes de IA (como ChatGPT, GitHub Copilot u otros) está permitido como herramienta de apoyo durante el desarrollo de la práctica, el diseño de la arquitectura y la depuración de errores. No obstante, **el uso acrítico de la IA (copiar y pegar código sin comprender su funcionamiento) penaliza gravemente el proceso de aprendizaje.**
- Tened en cuenta que todo lo implementado en esta práctica se alinea directamente con el temario oficial de la asignatura: desde la **comunicación entre componentes distribuidos (sockets)** y los **patrones basados en eventos y microservicios**, hasta la **escalabilidad**, los **patrones de resiliencia** y los aspectos de **seguridad**. En la defensa de la práctica y en el **examen de teoría** se exigirá un dominio absoluto de estos conceptos. Un alumno que simplemente haya delegado el código en una IA sin entender las tramas, la concurrencia o los estilos arquitectónicos no podrá superar las pruebas de evaluación individual ni las preguntas teóricas asociadas.
- La IA debe utilizarse exclusivamente como un apoyo para resolver bloqueos o acelerar tareas repetitivas, siendo responsabilidad absoluta de los estudiantes conocer, explicar y ser capaces de modificar cualquier línea de código de su repositorio sin ayuda externa.
- Los prompt usados en la IA para el desarrollo de esta deberán incluirse en Github y compartirse con el profesorado.

Los estudiantes deberán **entregar**, además, mediante la funcionalidad de evaluación del UACloud antes de la fecha establecida a su profesor de prácticas una **memoria de prácticas**, con el código fuente y compilados generados, así como un documento donde se detalle la siguiente información. El formato es libre, pero debe ser un documento ordenado y debidamente formateado, cuidando la redacción y ortografía.

- Portada con los nombres, apellidos y DNI de los estudiantes, año académico y el título de la práctica.
- Un informe donde se indique el nombre de los componentes software desarrollados y una descripción de cada uno de ellos, explicando y enviando además el código fuente de todos ellos.
- El detalle, paso a paso, de una guía de despliegue de la aplicación, que deberá ser la misma que utilice cuando haga la corrección de la práctica.
- Capturas de pantalla que muestren el funcionamiento de las distintas aplicaciones conectadas.

Cada profesor de prácticas podrá solicitar a los estudiantes cualquier otra evidencia que el profesor considere adecuada para poder formalizar la evaluación.

La fecha de entrega se realizará en la semana del 26 de octubre de 2026.